Challenge Skiplist

Student: Tommaso Tosca 10804131

1. Implementation and Expected Complexity

A randomized skip list was implemented in C17 to store a set of distinct signed 64-bit integer keys. Each node stores a key, a randomly generated height and forward pointers. Heights follow a geometric distribution with promotion probability $p = 1/2$. The maximum height is fixed at 32, which is sufficient for the tested sizes.

The implementation supports four operations: search, insert, ordered union and split.

Search and insert have expected $O(\log N)$ time complexity. Union and split also have expected $O(\log N)$ complexity, since they modify forward pointers without copying existing nodes. The original lists remain empty and reusable, while the resulting lists own the nodes. Expected space complexity is $O(N)$, since each node has a constant expected number of pointers.

2. Benchmarks

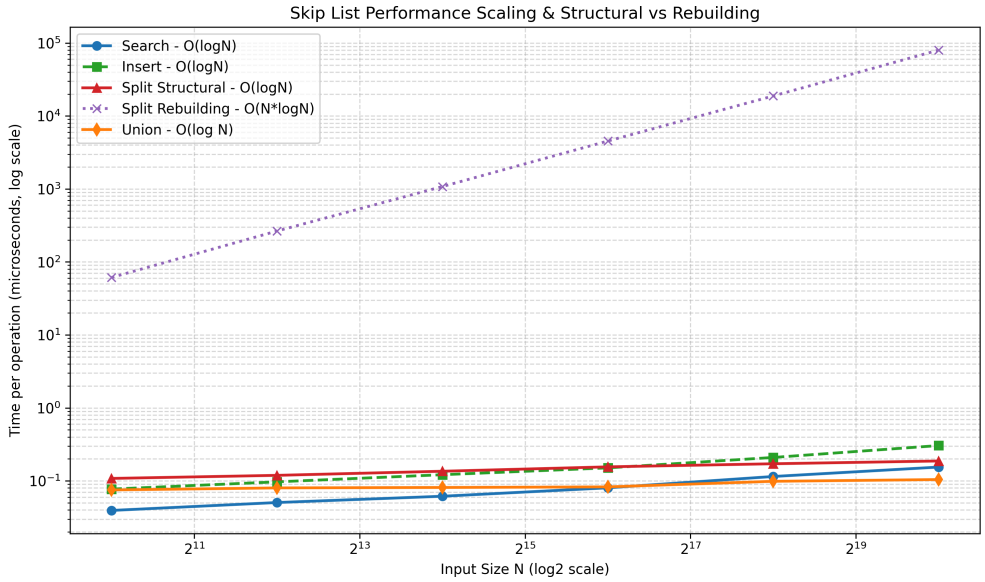
Search used 50% hits and 50% misses. Insert always used absent keys. Split used pivots near the middle, while union used two equally sized lists with disjoint ordered keys. For union, eight pairs of lists were alternated to avoid repeating the same traversal. Input construction and restoration were excluded from the measured time. Six input sizes from 1024 to 1048576 were tested, using three seeds (42, 234, 1000) and three timing repetitions per seed.

Environment: i7 14th, Windows 11, MSVC 14.44 (Visual Studio 2022), Release (/O2), Google Benchmark 1.10.0.

3. Experimental Results

The following results were obtained from the final benchmark run (raw_results.json).

Operation	Expected complexity	Time Google Benchmark fit	CPU Google Benchmark fit
Search	$O(\log N)$	Complexity: `5.94 lgN`, RMS: `28%`	Complexity: `5.85 lgN`, RMS: `28%`
Insert	$O(\log N)$	Complexity: `11.21 lgN`, RMS: `27%`	Complexity: `17.83 lgN`, RMS: `17%`
Structural union	$O(\log N)$	Complexity: `5.66 lgN`, RMS: `13%`	Complexity: `16.19 lgN`, RMS: `17%`
Structural split	$O(\log N)$	Complexity: `9.68 lgN`, RMS: `5%`	Complexity: `19.17 lgN`, RMS: `5%`
Rebuilding split	$O(N \log N)$	Complexity: `3.82 NlgN`, RMS: `3%`	Complexity: `7.65 NlgN`, RMS: `17%`



The measured results are generally consistent with the expected complexities. Union initially showed almost constant execution times when the same lists were reused. Alternating different lists produced a clearer logarithmic trend. In some runs, union and structural split were fitted as constant, while rebuilding split was fitted as linear. This may be due to fixed costs, such as memory allocation, randomized nature of skiplist, and the limited input range. The fitted models do not necessarily match the theoretical complexities.

4. Structural Split vs Rebuilding

Structural split only changes existing links and has expected $O(\log N)$ complexity. Rebuilding split creates new nodes using ordinary insertions, giving an expected cost of $O(N \log N)$. The timing difference is substantial: the structural implementation takes approximately 100 ns for 1024 elements and 190 ns for 1048576 elements, while rebuilding takes approximately 0.09 ms for 1024 elements and 124 ms for 1048576 elements.

Logical-work measurements: A traversal counts each move from one node to another through the pointer next[i]. Search and insert traversals increased from about 10 to 19, while structural union and split required between 6 and 28 traversals. Rebuilding split increased from about 9,000 to 20 million traversals. All traversal counts depend on the random seed. Structural split allocated no nodes, while rebuilding allocated N nodes.

5. Correctness and Limitations

Correctness checks: Tests were performed on empty lists, duplicate keys, searches, 64-bit boundary keys, and split-union operations. During 1,000 mixed operations for each of three seeds (42, 234, 1000), the content of the skiplist was checked against a sorted array. AddressSanitizer was also used, and no memory errors were detected. One limitation is that execution times depend on the generated structures and the execution environment, so the fitted complexity does not always match the theoretical one. Overall, the results support the expected complexity of structural operations and show that structural split is much faster than rebuilding.

References

1. Course lecture on randomized data structures.
2. Google Benchmark documentation.
3. W. Pugh, *A Skip List Cookbook* (1990).